

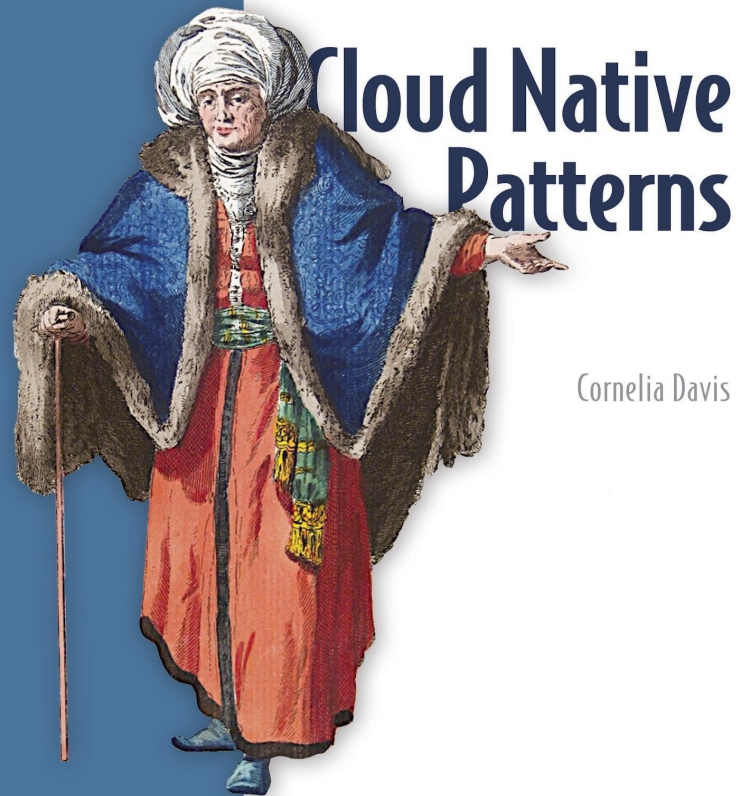
Building Durable, Production-ready Agents with OpenAI Agents SDK and Temporal



May 2026
Replay

- Developer (wasn't Ops)
- Web architectures for 20+ years
- Cloud-native for more than a decade
- Cloud Foundry for 8+ years
- Kubernetes for nearly 10
- Author 🙌

Developer Advocate, Temporal





Melissa Herrera

Developer Advocate @ Temporal

Dog Mom & Matcha Lover @ Life



@MelGoesTech



/herrera-melissa



AGENDA

- ✓ Foundations of Agentic Systems
- ✓ Building a Durable Agent from Scratch
- ✓ Building with an Agent Framework, but Durable
- ✓ Adding MCP Tools
- ✓ Humans in the Loop (HITL)
- ✓ Orchestrating (micro)Agents



Exercises/Demos

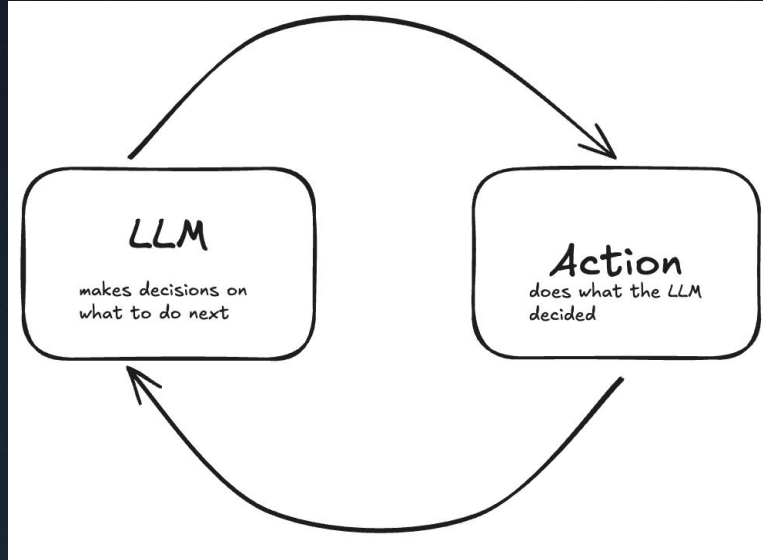


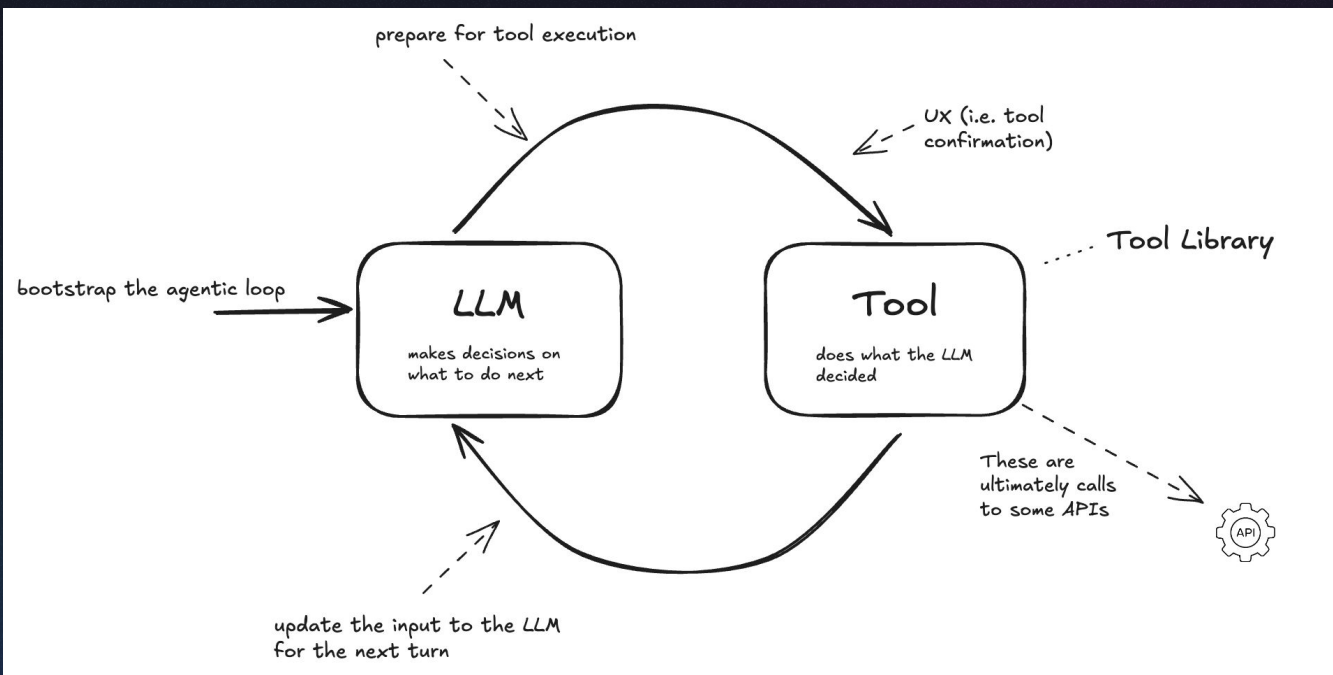
<https://t.mp/ai-workshop-python>

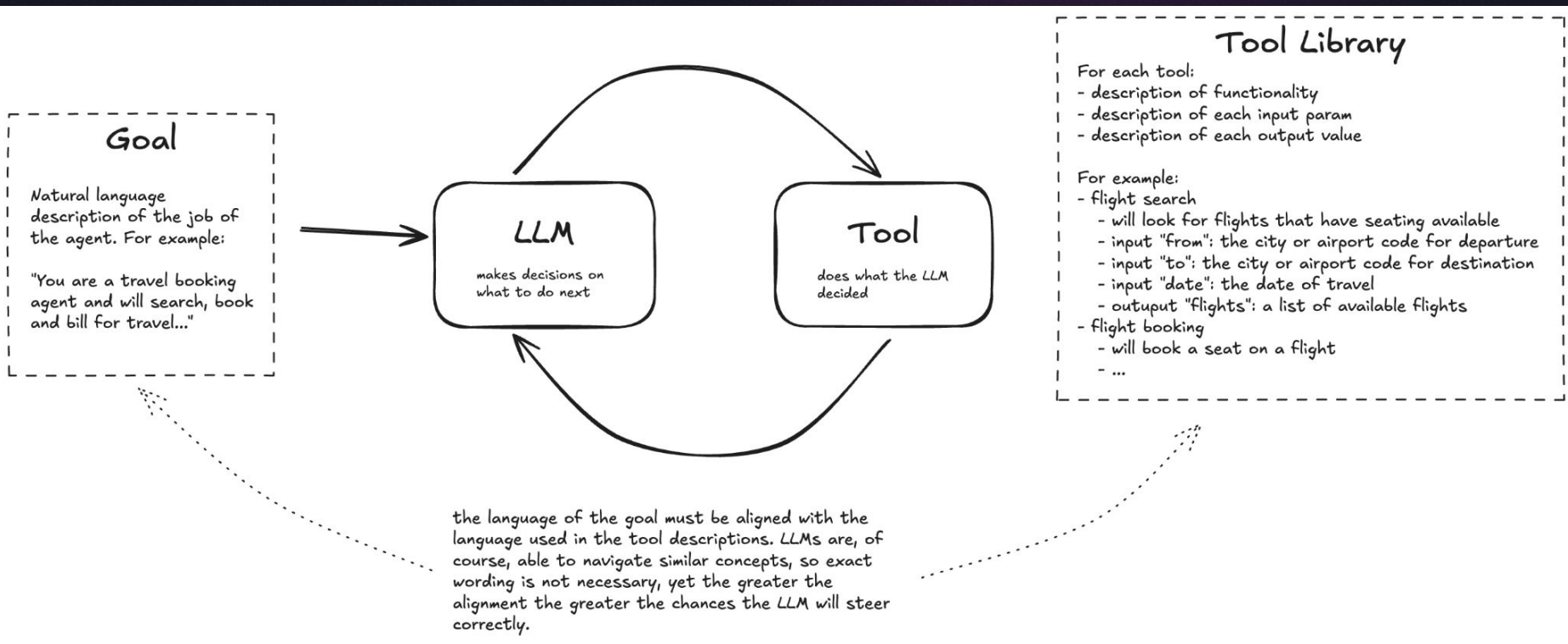


Foundations





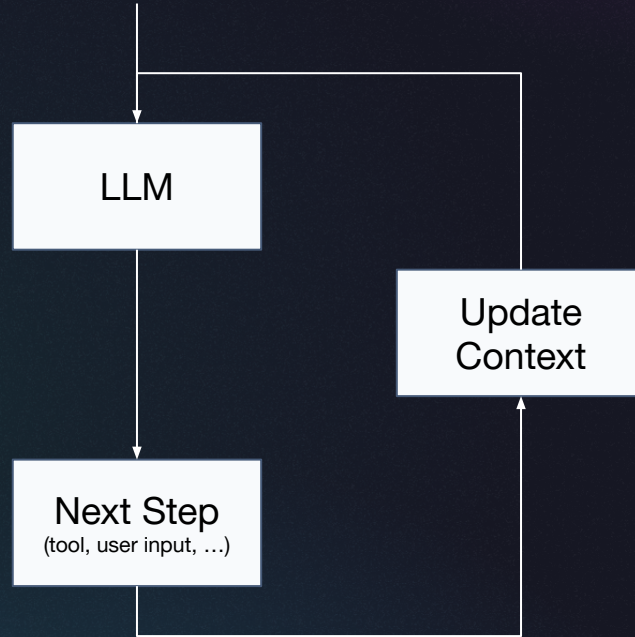


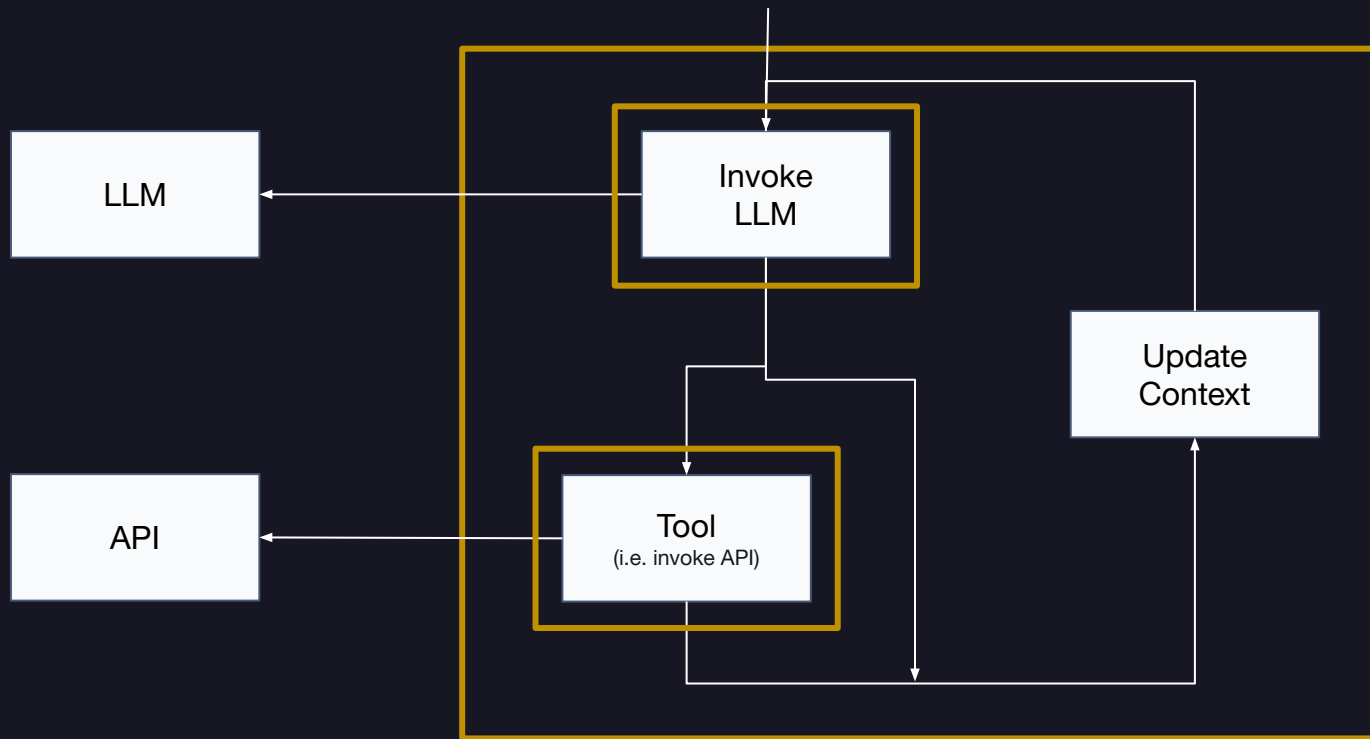


Building a **Durable** Agent from Scratch



Basic *agentic* loop





Introducing Temporal

- Technology and open source project that delivers resilience for distributed systems in a novel way.
- Supports a programming model that allows developers to code the **happy path**, while the platform provides services that compensate for a wide range of distributed system failures.
- Platform comes in the form of a service + SDKs

SDK is available for Go, Java, Python, PHP, Typescript, .Net, Ruby



Temporal

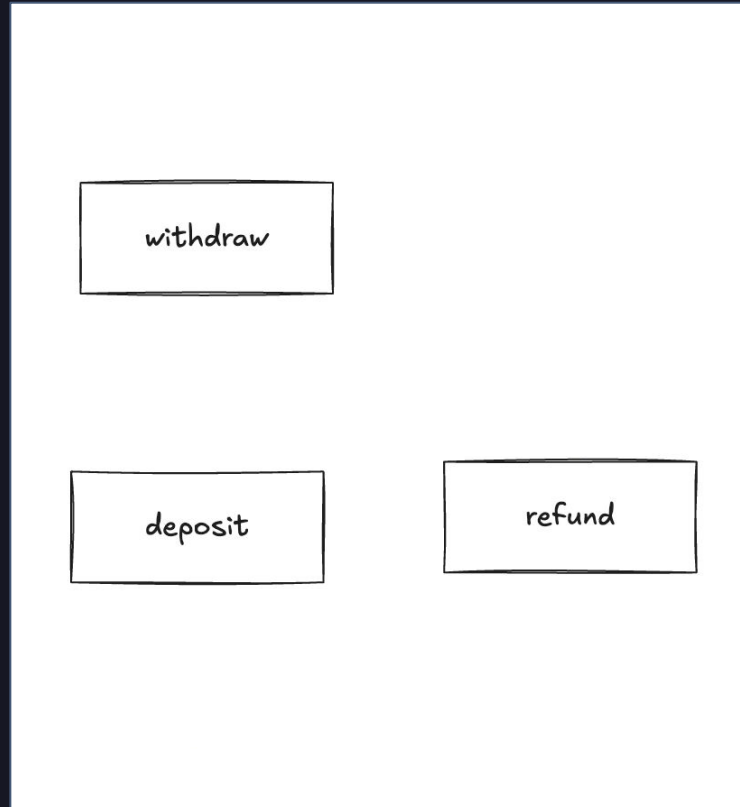
Note: The OpenAI Agents SDK + Temporal solution is currently only available for Python. Typescript coming in the future.

The MIT License

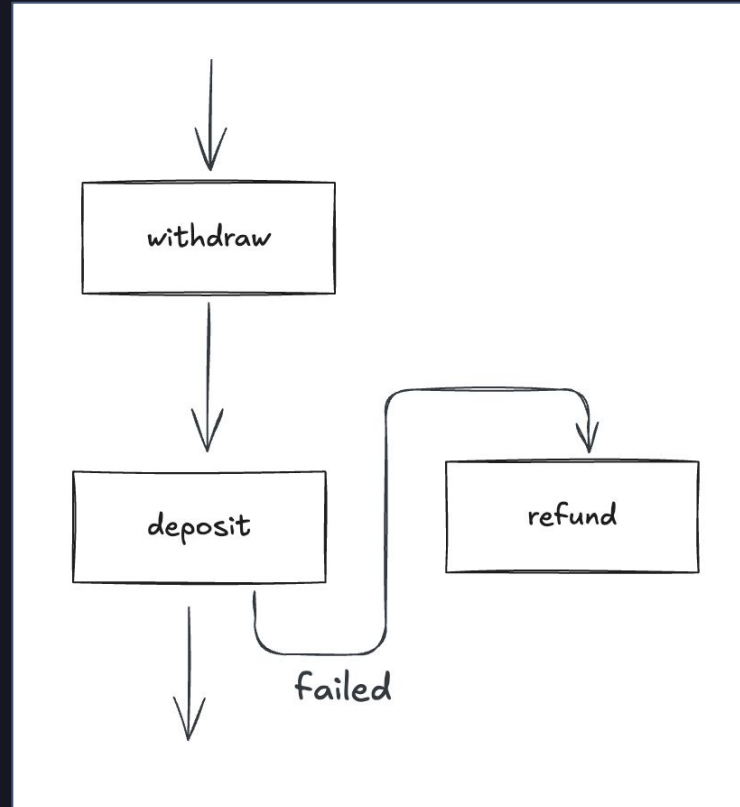
Copyright (c) 2020 Temporal Technologies Inc. All rights reserved.

Copyright (c) 2020 Uber Technologies, Inc.

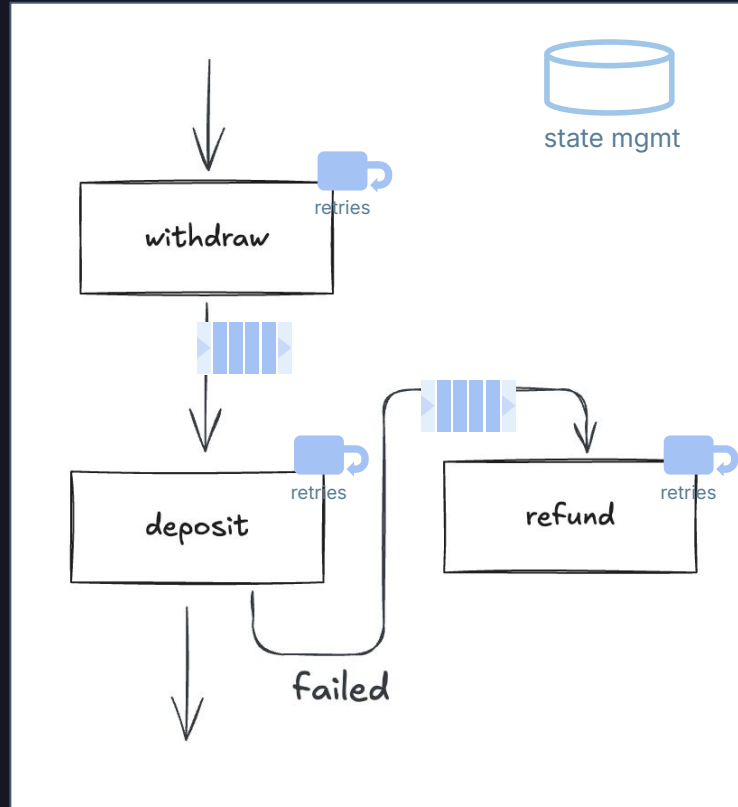
Temporal Activities



Temporal Workflows

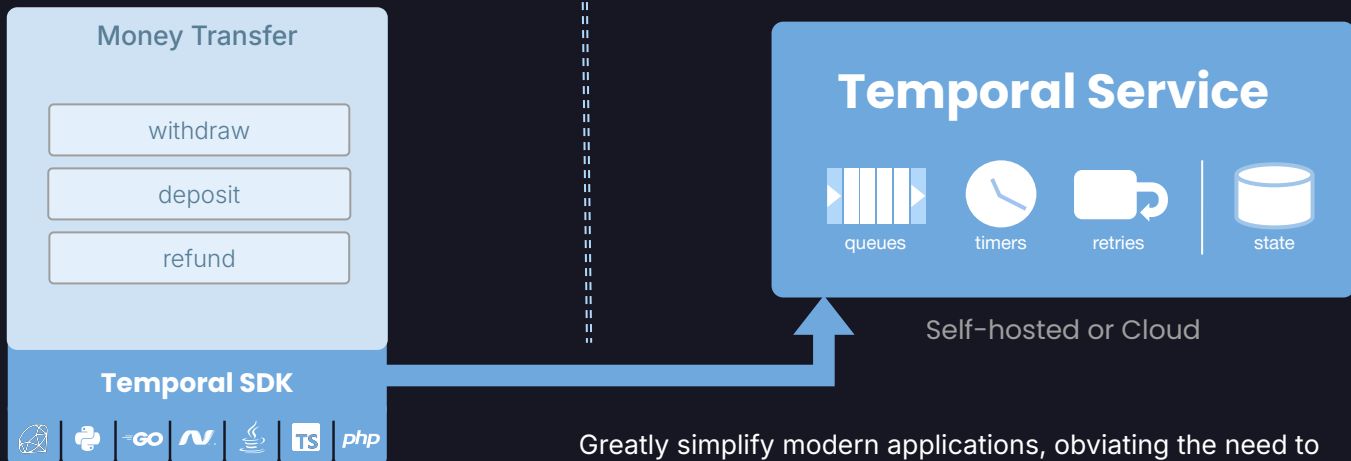


Temporal Workflows and Activities = ✨

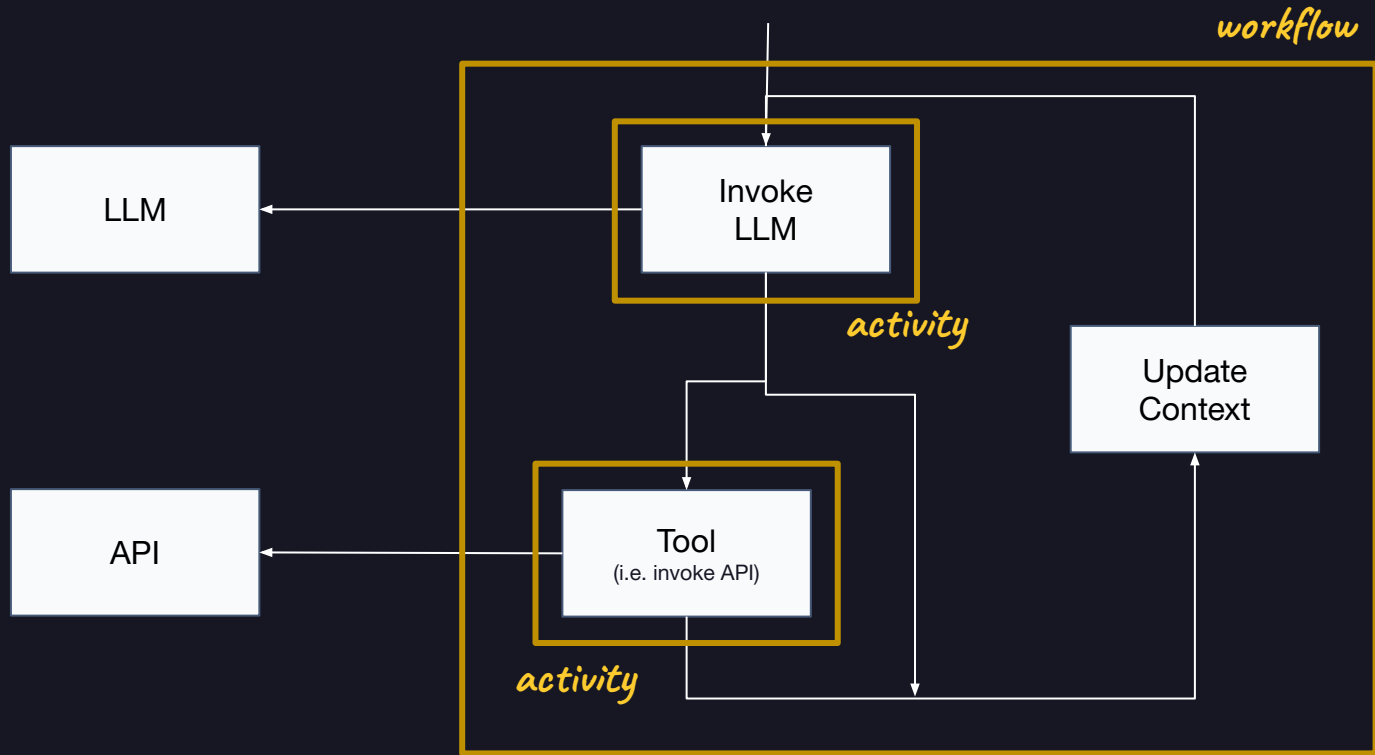


ENTER DURABLE EXECUTION

Define a workflow and then
maintain state and coordinate execution

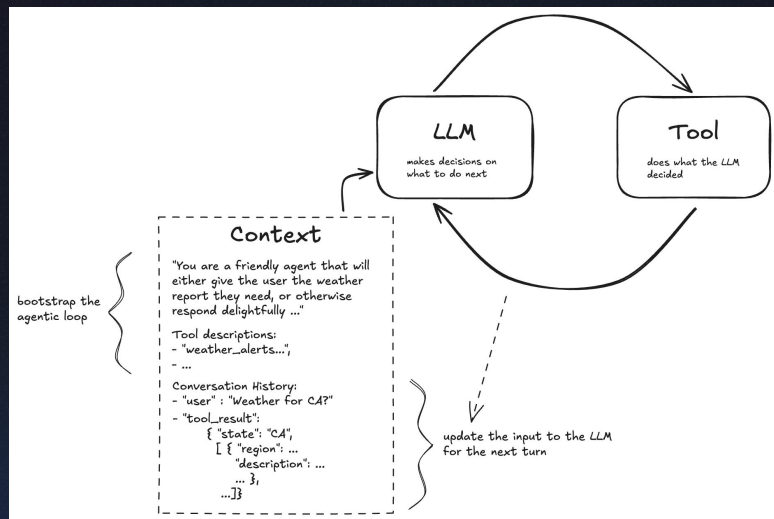


Greatly simplify modern applications, obviating the need to create queues, manage timers, publish and consume events, implement retries and rollbacks, checkpoint state, and more.



Agent implementation= (recoverable) orchestration process(es) (agentic loop)
+ LLM invocation process(es)
+ tool process(es)
(it's a distributed system)

Agent: Native Temporal Implementation



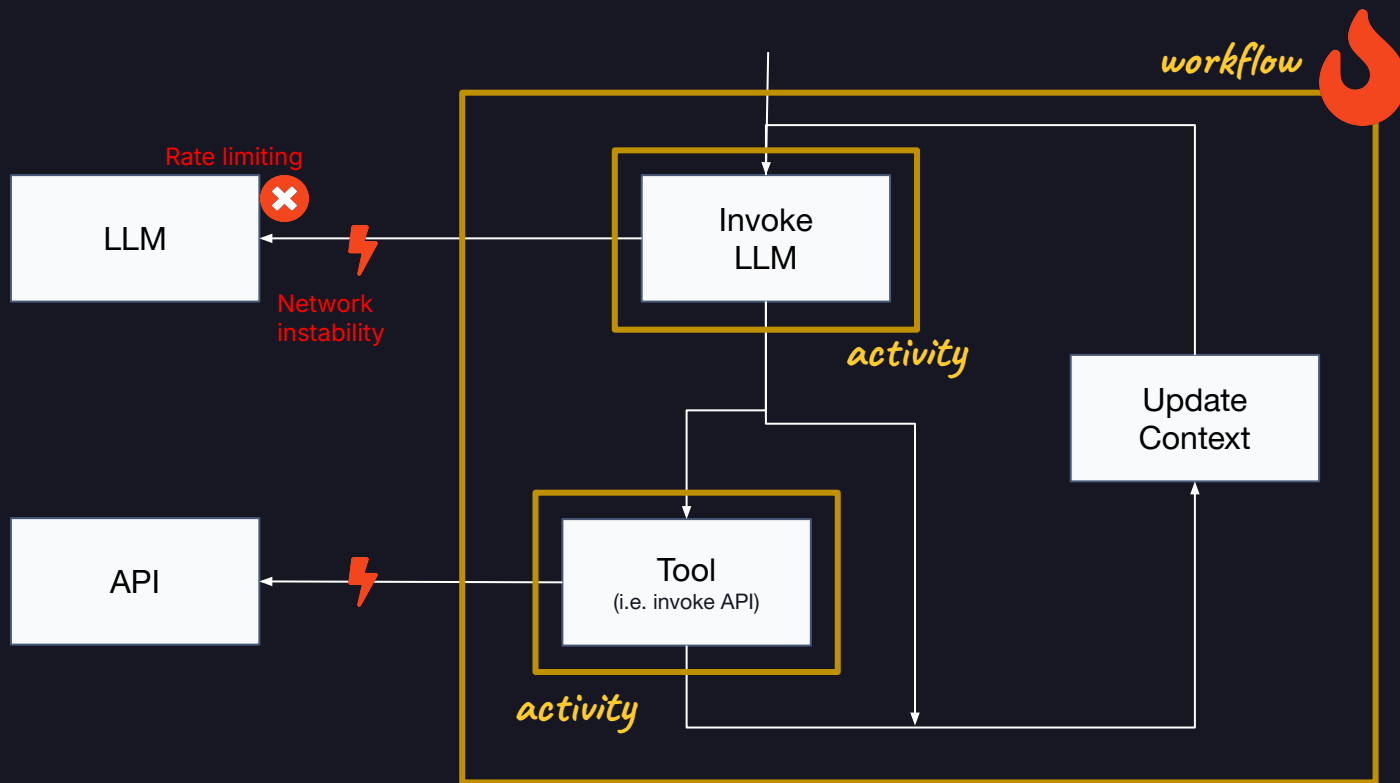
- System Prompt - defines the job of the agent
- LLM **activity**
- Tool calling **activity**
- Agentic loop **workflow**
- Tool definitions
 - Loosely coupled from the agent
 - Must craft or generate tool definitions sent to LLM
 - (generating from function definitions preferred)
 - Virtually all model APIs allow for callables to be passed in, but we need to serialize



DEMO #1

agentic loop with tool calls





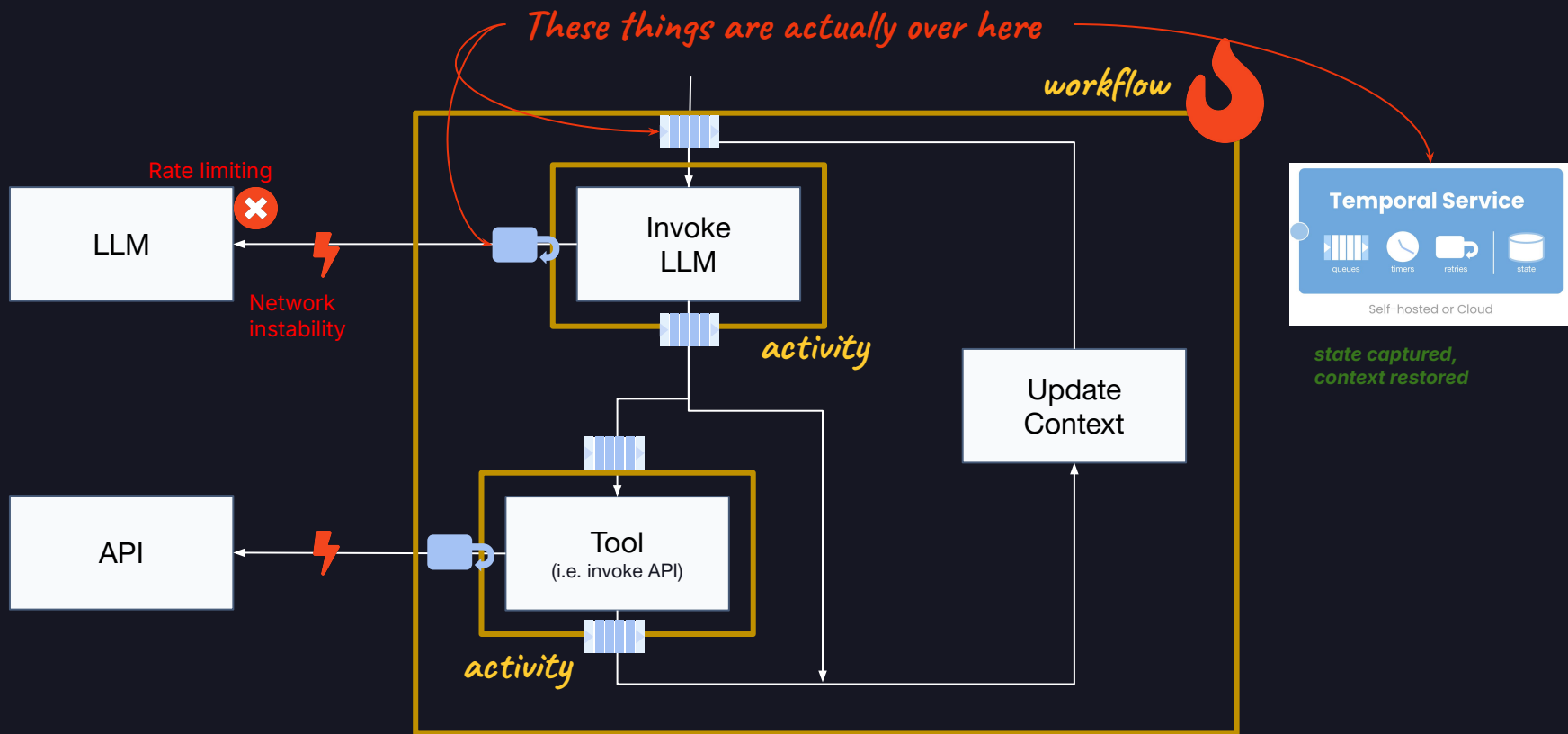
Agent implementation= (recoverable) orchestration process(es) (agentic loop)
+ LLM invocation process(es)
+ tool process(es)
(it's a distributed system)

continued

DEMO #1

agentic loop with tool calls





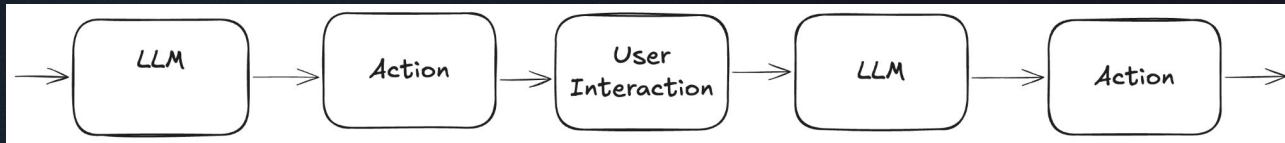
Agent implementation= **(recoverable)** orchestration process(es) (agentic loop)
+ LLM invocation process(es)
+ tool process(es)
(it's a distributed system)

Things to note:

- By default, most model APIs will retry LLM invocations (for transient failures) - **turn them off!** << We want Temporal to handle retries
- Most model APIs support passing of tools as callables but because tools are invoked via activity - **we need to serialize:**
- Increasingly, LLM invocation APIs will call tools. **Turn that off** - we want tools to be called via Temporal activity:
- Most LLM APIs support [Structured Outputs](#); for tools this is automatic



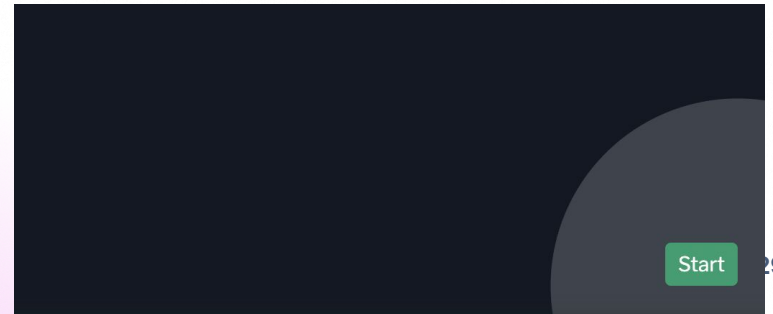
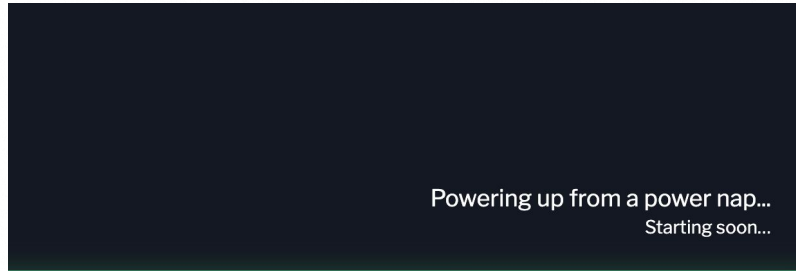
IMPORTANT: A lot of value in **Fixed Flow**



Let's do some playing...

t.ly/fmXNE

When it's starting up...



t.ly/fmXNE

instruct

Building Durable AI Agents with Python and Temporal

Overview

Invite details

Progress

Hide Instructions

🕒 4h

Instructions

View notes

Demo 1: The Hand-Written Agentic Loop

Demo 1: The Hand-Written Agentic Loop

What you're looking at

Click the **Editor** tab and open `demo1-agentic-loop`. The key files are:

- `workflows/agent.py` - the `while True` loop: call LLM, dispatch tool if needed, repeat until the model returns a final answer
- `activities/openai_responses.py` - the LLM activity: sends the conversation history to OpenAI and returns the model's response
- `activities/tool_invoker.py` - a single *dynamic* activity that routes to whichever tool the LLM chose
- `tools/` - four weather tools: `get_ip_address`, `get_location_info`, `get_coordinates`, `get_weather`

Run it

Worker tab - start the worker:

📋 copy

```
uv run python -m worker
```

Skip

Check

>_ Worker

>_ Starter

🔗 Temporal UI

🔗 Network Control Panel

🔗 Editor

🔄

```
root@workshop-host: /workspace/workshop/demo1-agentic-loop#
```



t.ly/fmXNE

>_ Worker

>_ Starter

[Temporal UI](#)

[Network Control Panel](#)

[Editor](#)

 Web UI

 Namespaces

 Workflows

 default



 local 



0 Workflows

 Refresh

 Start Workflow

May 4, 2026, 1:00:57.55 PM PDT



t.ly/fmXNE

>_ Worker >_ Starter Temporal UI Network Control Panel Editor

Network Control Panel

Toggle external service access for the workshop demos

EXTERNAL SERVICES

- **Geolocation**
(ip-api.com) ☒
- **IP Info**
(icanhazip.com) ☒
- **OpenAI**
(api.openai.com) ☒
- **Weather**
(api.open-meteo.com) ☒

KILL SWITCH

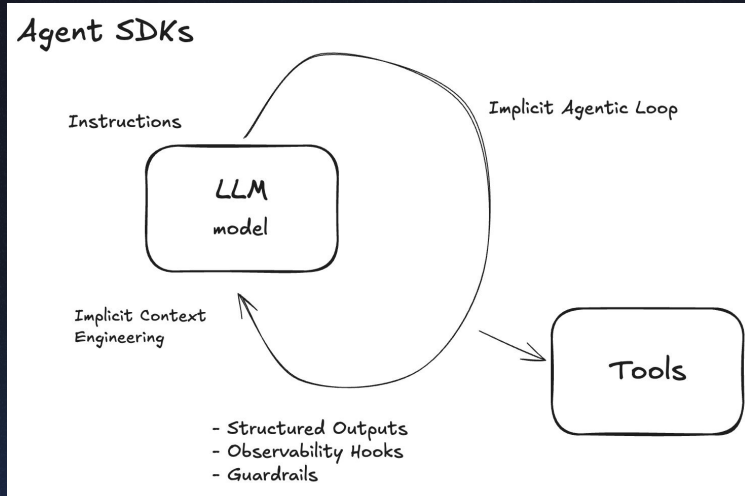
Kill Switch: Network is OPEN ☒
All external services are flowing normally



Building with an
Agent Framework
... but **Durable!**



Agent SDKs



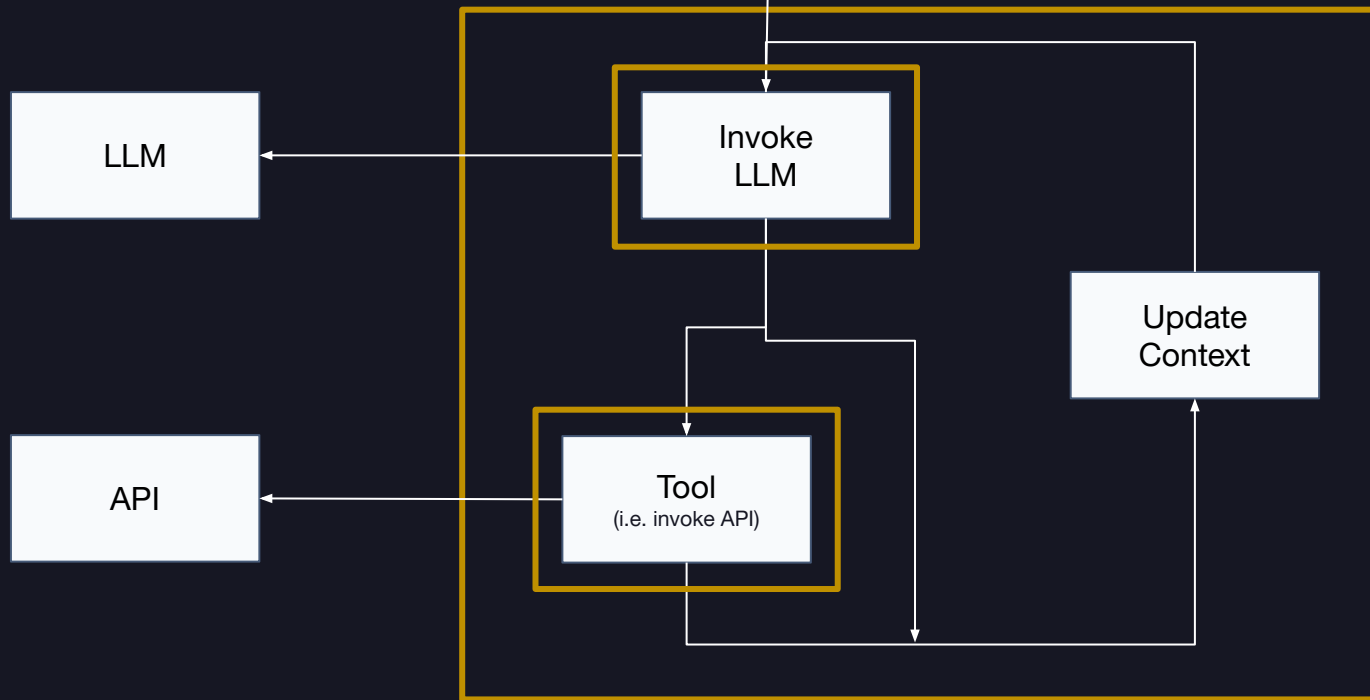
- At the highest level most agent SDKs are very similar.
- But their semantics do differ. I.e. Handoffs (something in the OpenAI Agents SDK)
- These are libraries



Without Temporal, you are responsible for scaling this whole thing



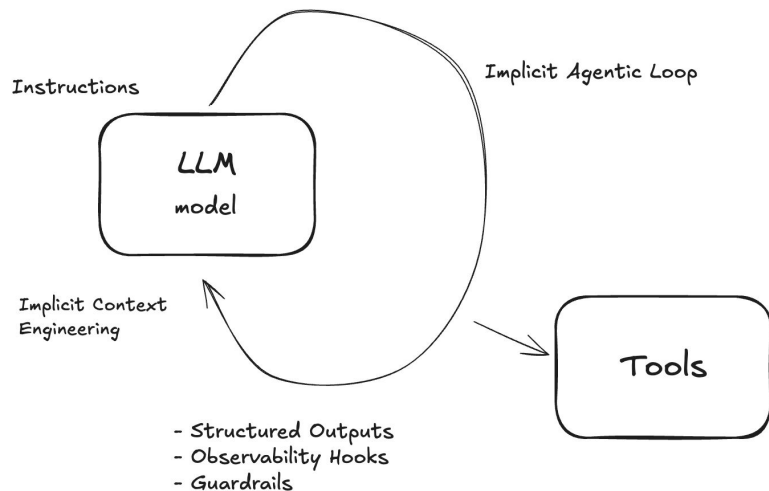
With the Temporal integrations, we decompose and scale it for you (in addition to the durability)

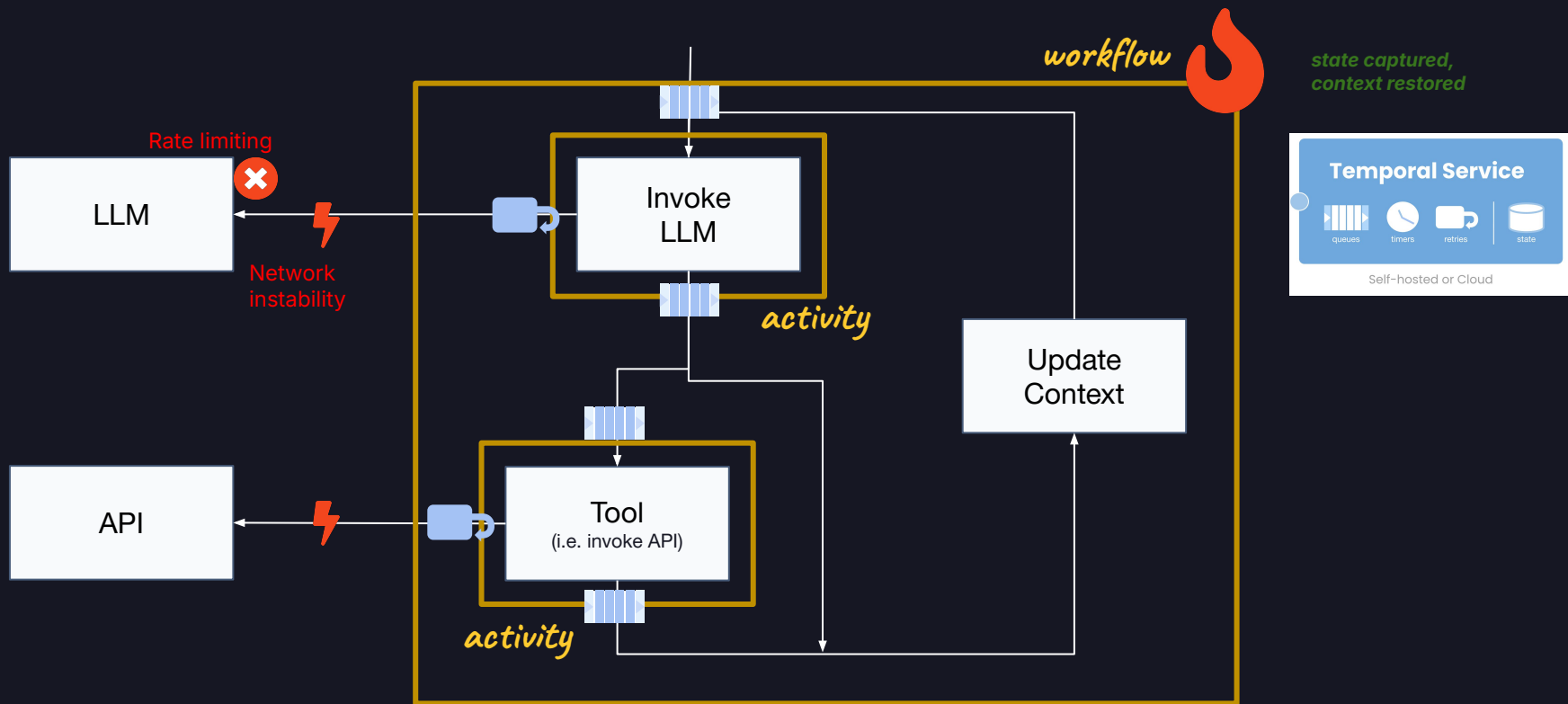


Open AI Agents SDK

```
1 agent = Agent(  
2     name="Agent",  
3     instructions=SYSTEM_PROMPT,  
4     model="gpt-4o",  
5     tools=[get_ip_address, get_location_info, get_weather])  
6  
7 result = await Runner.run(agent, input=question)  
8 return result.final_output
```

Agent SDKs





Agent implementation= **(recoverable)** orchestration process(es) (agentic loop)
 + LLM invocation process(es)
 + tool process(es)
(it's a distributed system)

DEMO #2

Agent Framework: From Non, to **Durable**

TBD



Things to note:

For OpenAI Agents SDK

- We overrode the **Runner** class
 - ([PR](#) to make it abstract)
- Use **activity_as_tool** to deliver an activity invoker proxy to the Agent (see [here](#))
- Use **nexus_operation_as_tool** to deliver a Nexus client proxy to the Agent (see [here](#))
- Built in tools (i.e. WebSearch)

Different agent framework integrations following different approaches



Different SDKs

Model APIs

- Primarily about invoking the model
 - Model
 - Tools
 - ...
- Most of these APIs do retries (turn them off!)
- Increasingly, they are doing more:
 - i.e. Invoking tools (Gemini)

Agent SDKs

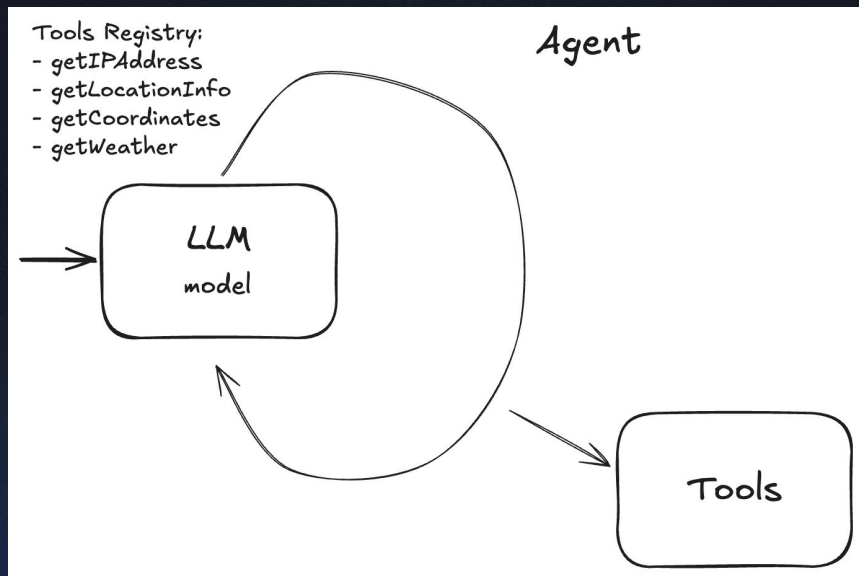
- Implement the agentic loop
- Default context mgmt (conversation history)
- Includes LLM invocation
 - Model
 - Tools
 - ...
- Turn off retries here too!
- Sessions - reentrant loops (i.e HITL)
 - but NOT durable
- Temporal delivers durability & turns them into distributed systems



MCP Tools



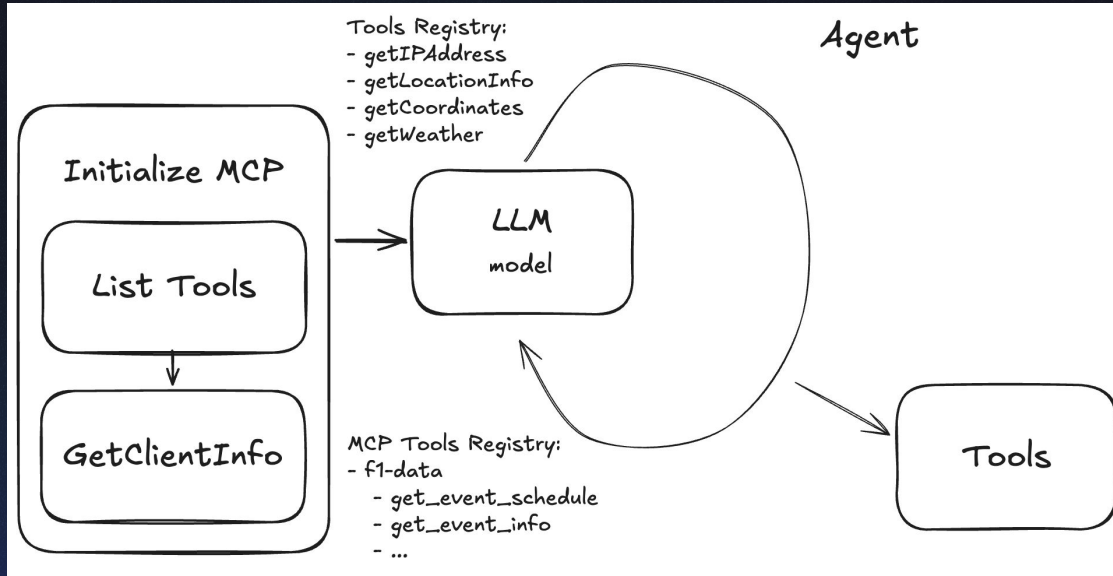
What we have so far:



- We've implemented our own tools registry
- The Agent Framework/Temporal integration delivers:
 - LLM invocations as activities
 - Tools invoked as activities**therefore durable!**
- Distributed implementation
- We provide APIs to serialize tool definitions



MCP adds:



MCP protocol implementation:

- MCP tools registry
- Tool definitions formatted for LLM

From the integration:

- Protocol actions as activities
- Tools as activities



MCP Overview

Primitives

- Prompts
- Resources (i.e. files)
- Tools (i.e. Agent-ready APIs)

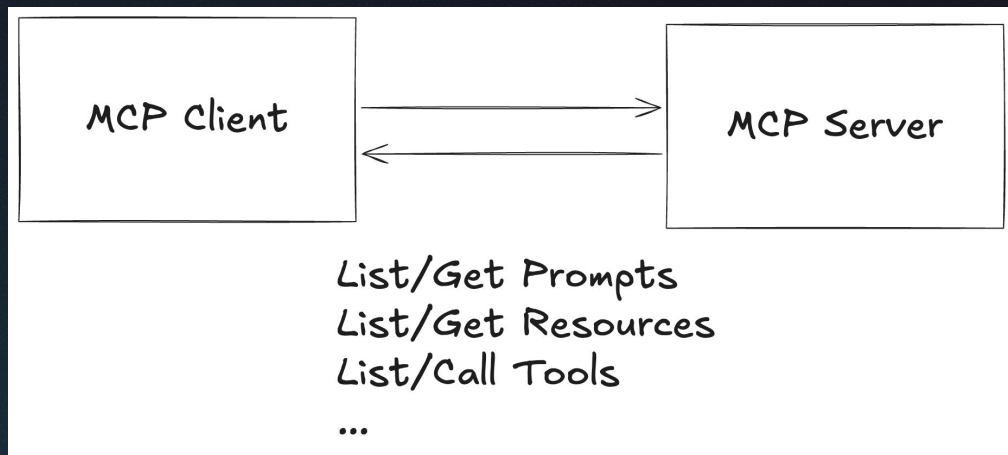
Client Server Architecture

- MCP Clients
- MCP Servers
- A transport protocol between them

* Note: there are a few other concepts, but these are the core ones



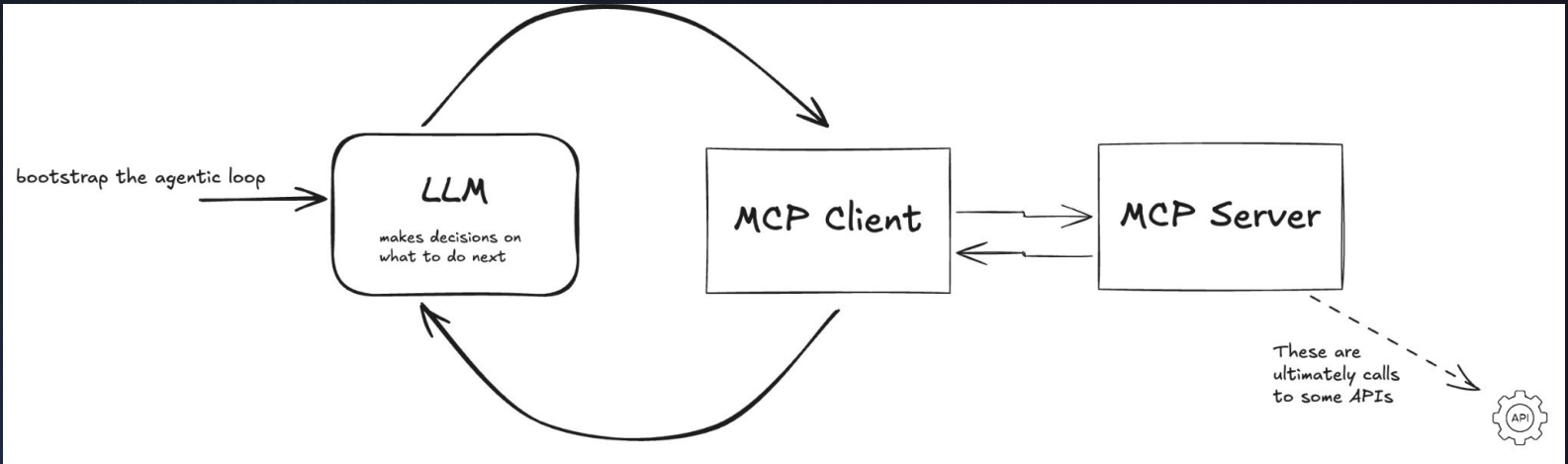
MCP Client/Server Architecture



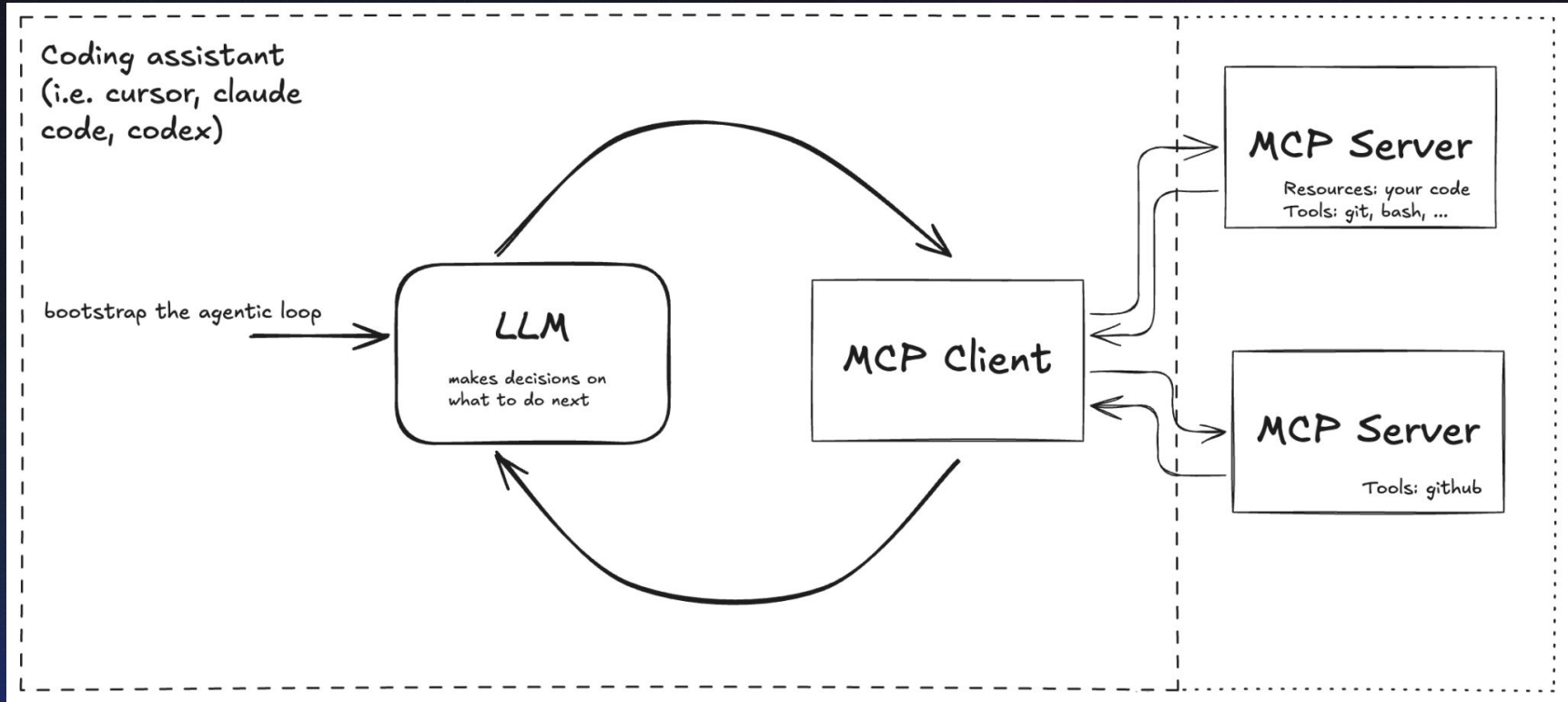
* Note: there are additional operations including notifications (one way), and sampling (Request/Response in the other direction).



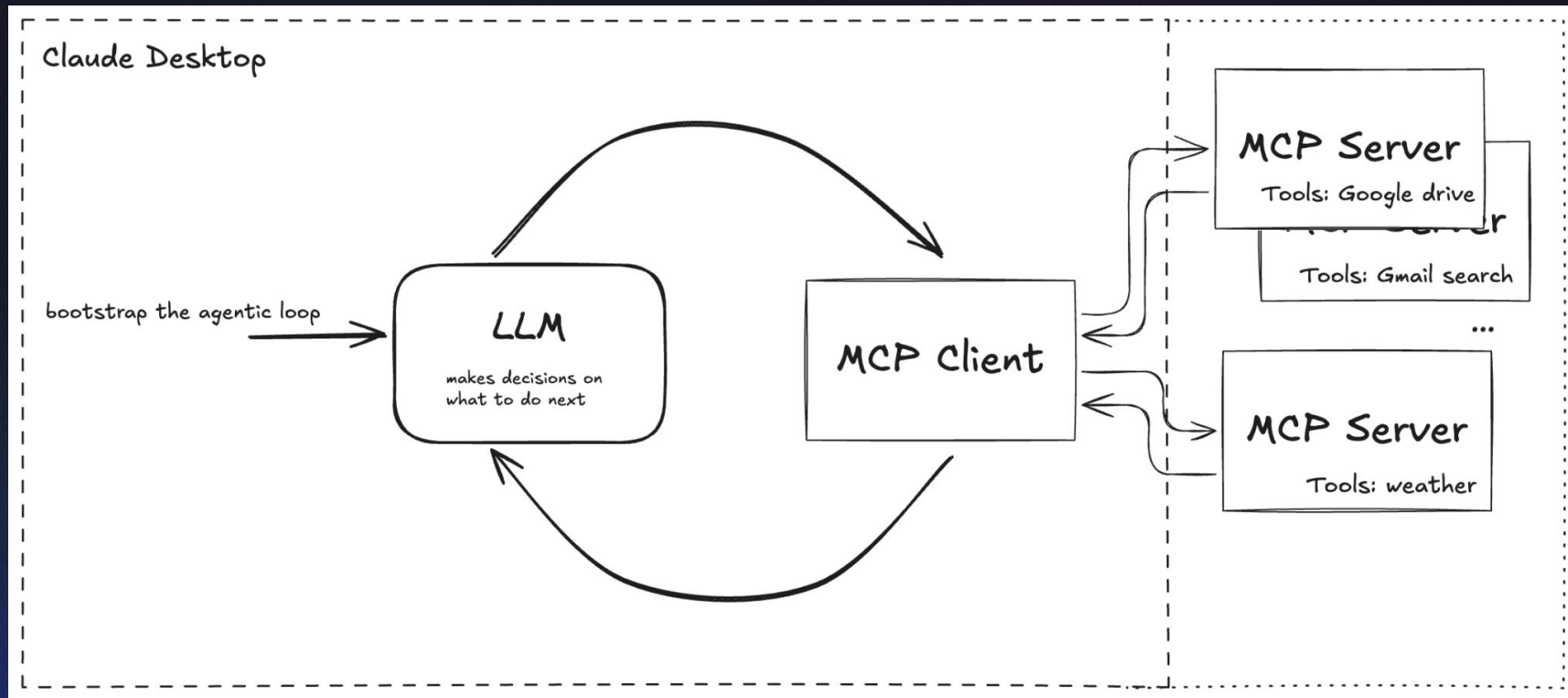
MCP Clients are Embedded in the Agent



Many Types of Agents Embed MCP Clients

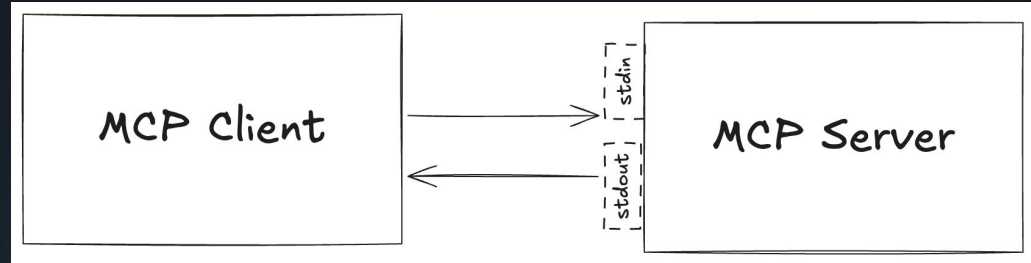


Many Types of Agents Embed MCP Clients

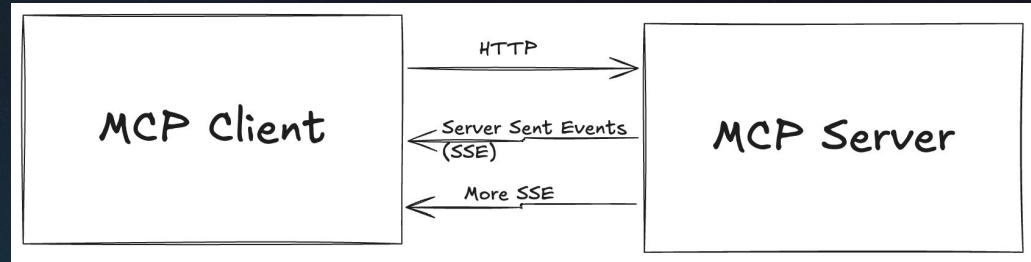


Supports Multiple Transport Protocols

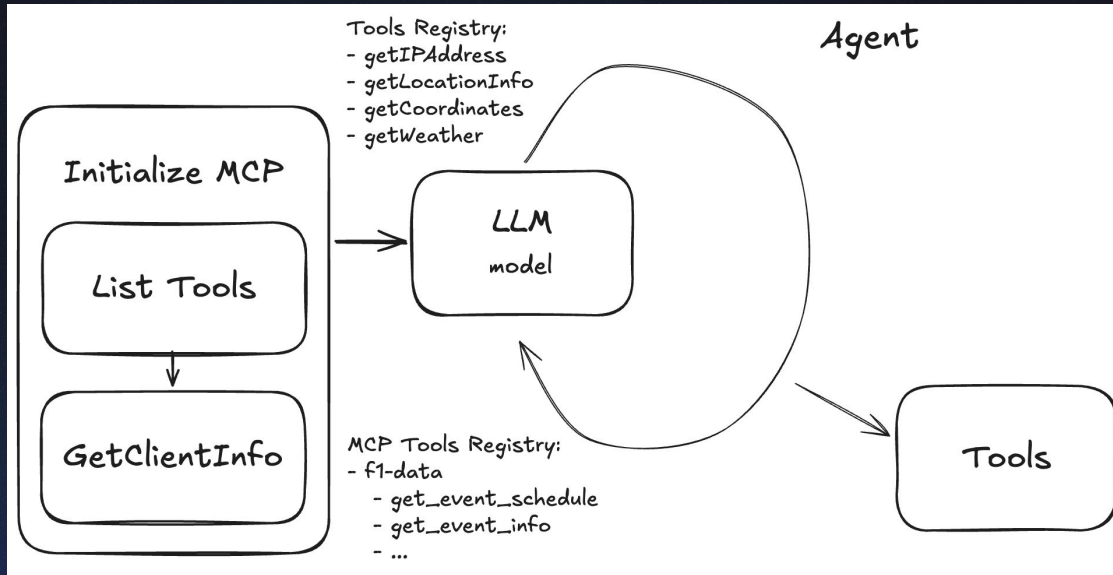
Transport: stdio



Transport:
streamable-http



MCP adds:



MCP protocol implementation:

- MCP tools registry
- Tool definitions formatted for LLM

From the integration:

- Protocol actions as activities
- Tools as activities



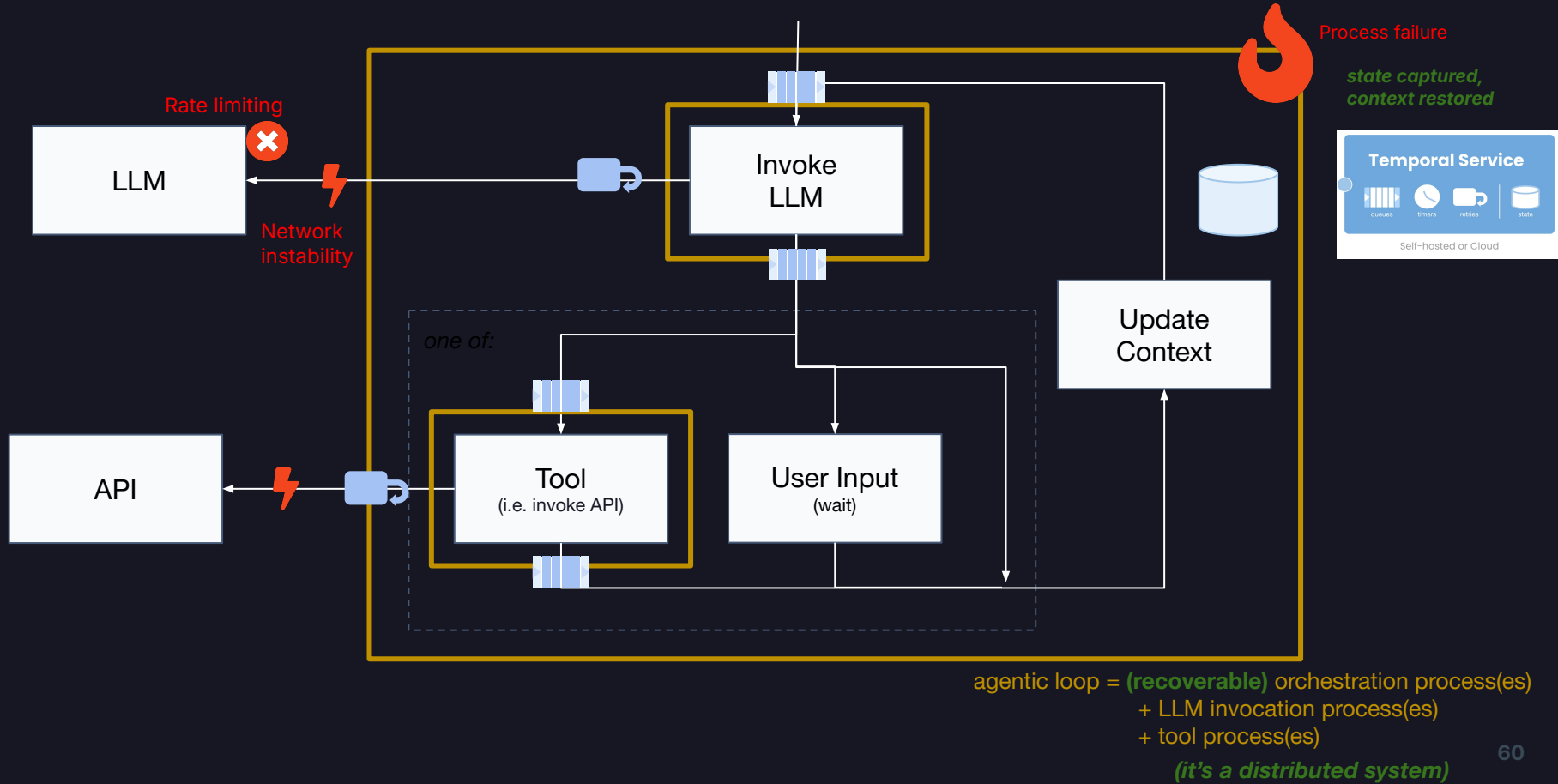
DEMO #3

MCP Tools



HITL





DEMO #4

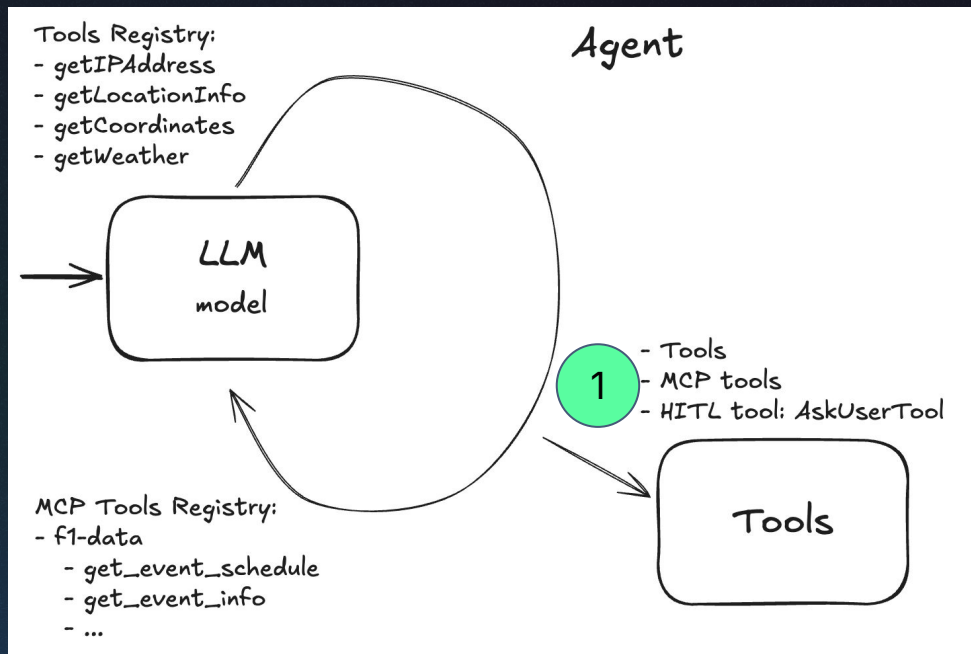
Durable and Resource efficient HITL

TBD

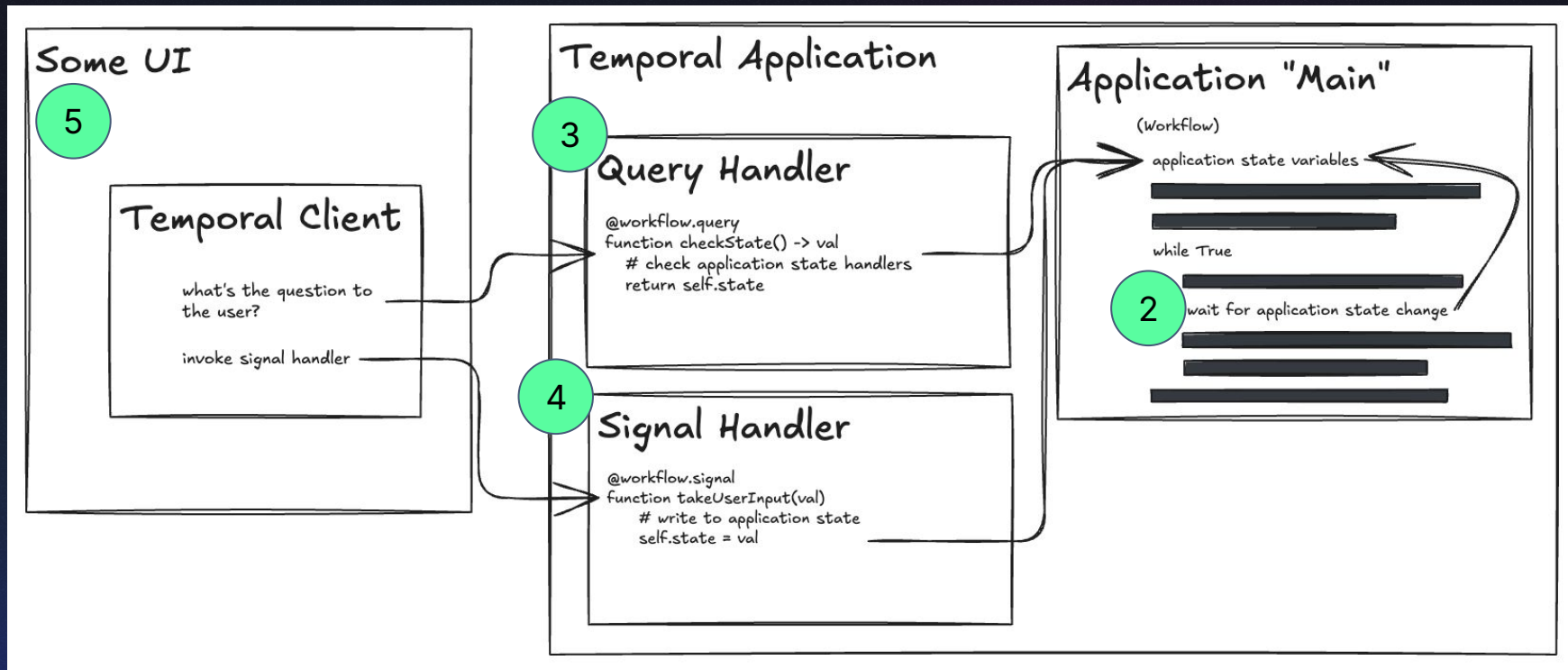


Overall architecture - For the LLM

That's Right! We give it a tool!

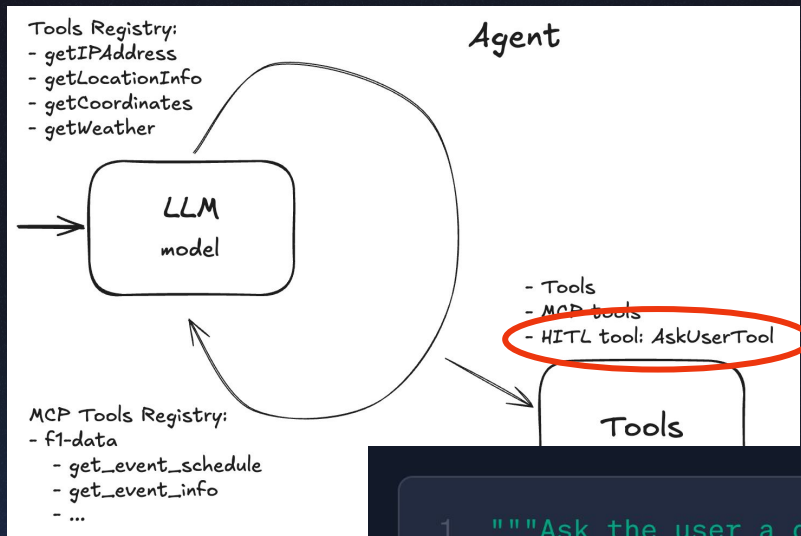


Overall architecture



1

Supply LLM with the Tool Description

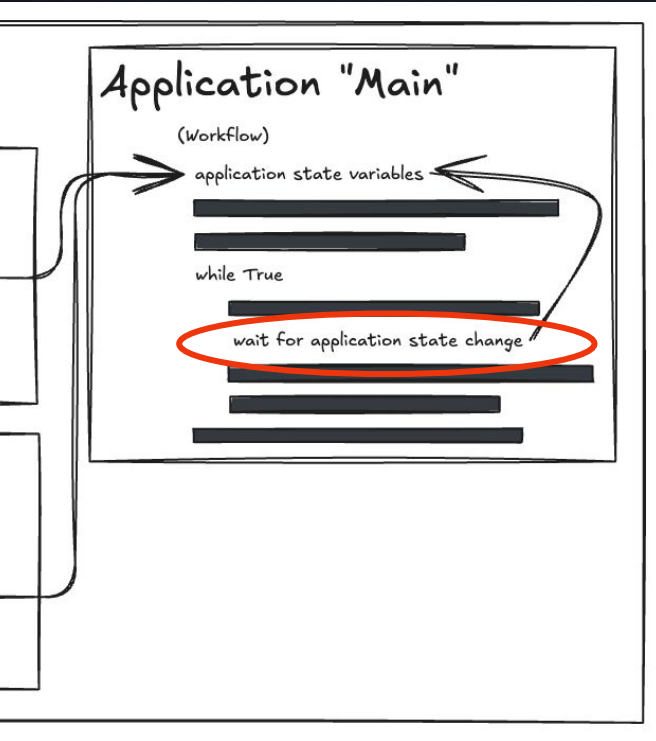


```
1 """Ask the user a question when you need clarification or more information.
2
3 Args:
4     question_text: The question to ask the user.
5 """
```



2

Wait for Input



Within the Workflow:

- State variables
- Tool runs in the workflow "thread"
 - Giving access to the state variables
- Wait for the user happens within the Tool implementation
 - This blocks the execution but...
- ... Does not lock up threads because with Temporal, an "await" ends the workflow task, freeing up the "thread" to be used for another task.



3

4

Query & Signal

Temporal Application

Query Handler

```
@workflow.query
function checkState() -> val
# check application state handlers
return self.state
```

Signal Handler

```
@workflow.signal
function takeUserInput(val)
# write to application state
self.state = val
```

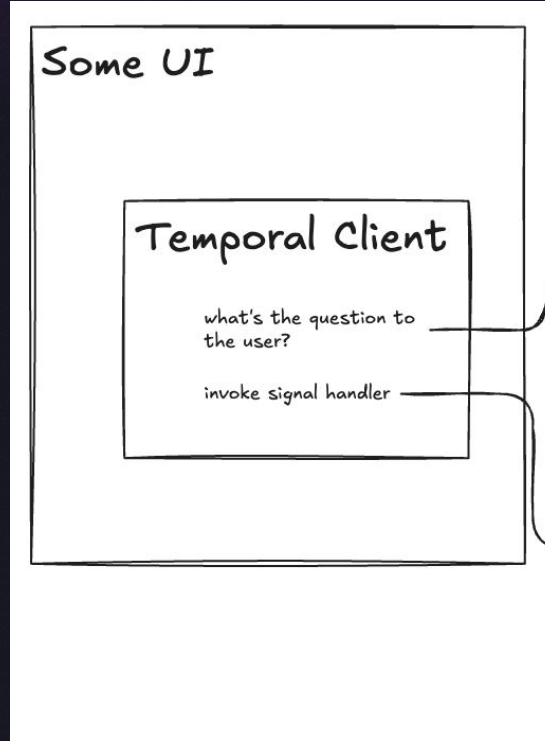
Within the Workflow:

- The Query Handler will check if there are any open questions
 - These have been provided by the LLM...
 - ... written into the workflow state via the **AskUser** tool
- The Signal Handler will provide a response to that question
 - Which the agent will write into the conversation history for the next turn



5

The Client

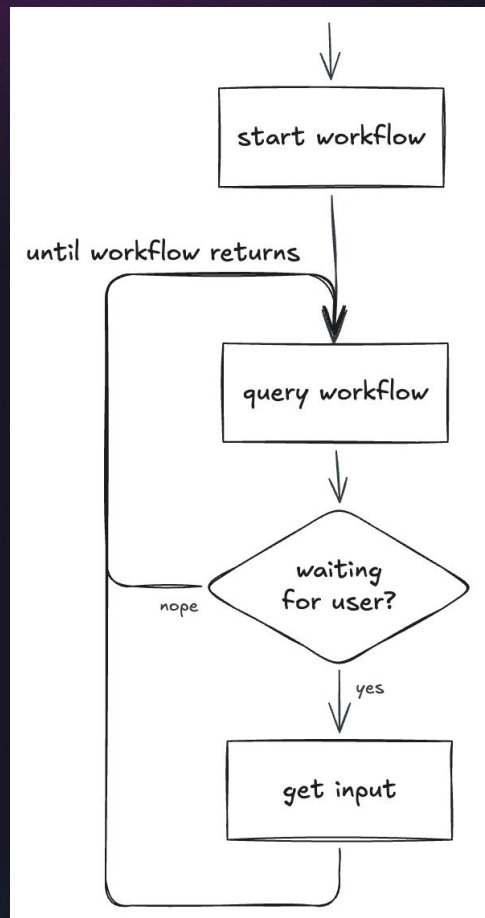


5

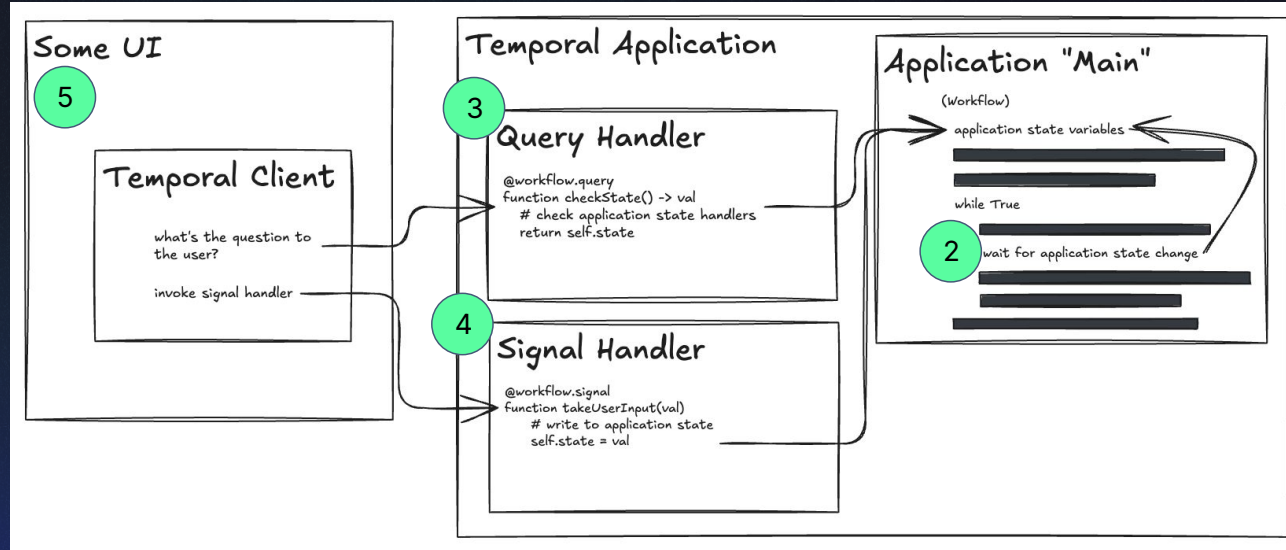
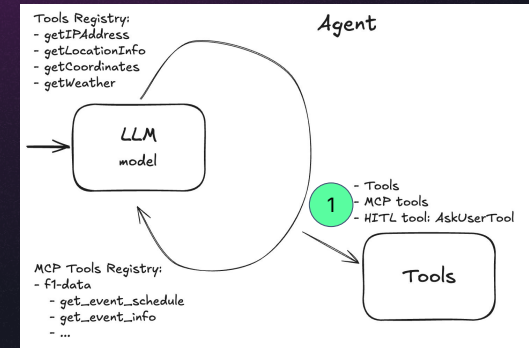
The Client

Please note:

- The client $\leftrightarrow$ workflow protocol may vary
 - We've implemented a polling protocol here but others are possible
 - i.e. MCP tasks
- The implementation here is naive - will be looping (and consuming resources) for a potentially long wait time
- Client can reconnect to running agent - state is preserved



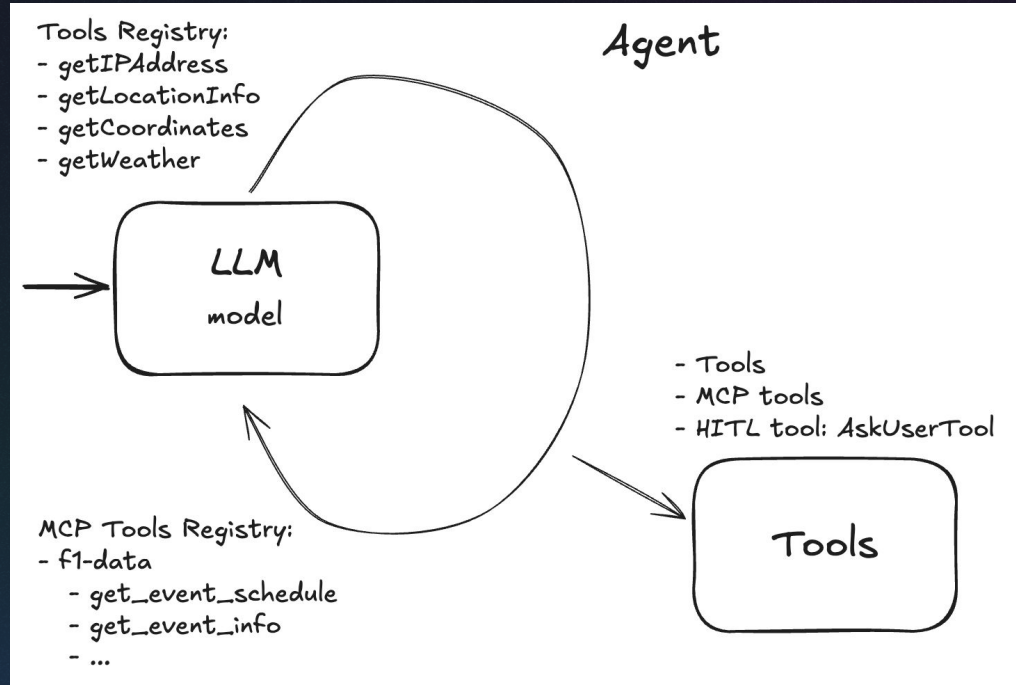
Overall architecture



Orchestrating (micro)Agents



So far, we've built a single, very busy agent...



Reasons for breaking things up

- Different teams
 - Different release cycles
 - Different tech stacks
- Modular architecture
 - Bulkheads, scale out, ...
- Scoping agents
 - Managing context
- ...

How Contexts Fail...



Context Poisoning

When a hallucination makes it into the context



Context Distraction

When the context overwhelms the training



Context Clash

When parts of the context disagree



Context Confusion

When superfluous context influences the response



...And How to Fix Them



RAG

Add relevant information to assist the LLM



Context Quarantine

Isolate subtask contexts in dedicated threads



Context Summarization

Boil down an accumulated context into a summary



Tool Loadout

Select only relevant tool definitions



Context Pruning

Remove irrelevant or unneeded information



Context Offloading

Store information outside the LLM's context

<https://www.dbreunig.com/2025/07/24/why-the-term-context-engineering-matters.html>

We've seen this before!

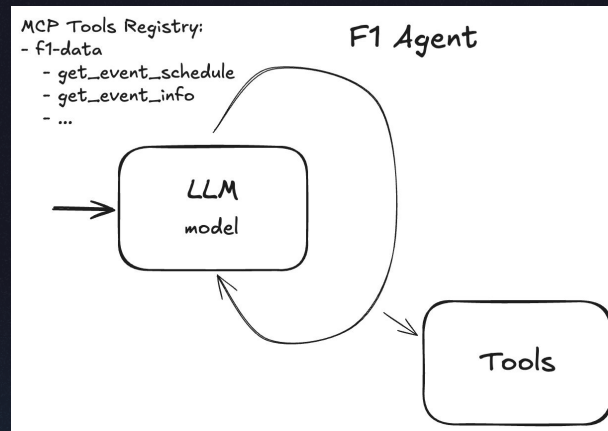
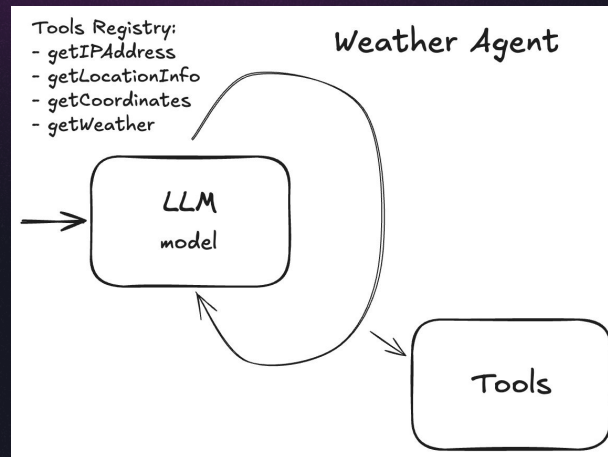
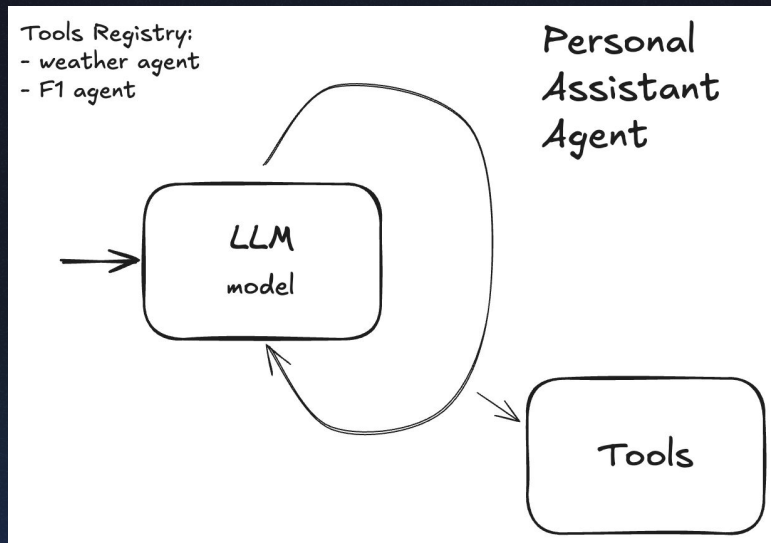
- Different teams
 - Different release cycles
 - Different tech stacks
- Modular architecture
 - Bulkheads, scale out, ...
- Scoping agents
 - Managing context
- ...

Blog



<https://t.mp/micro>

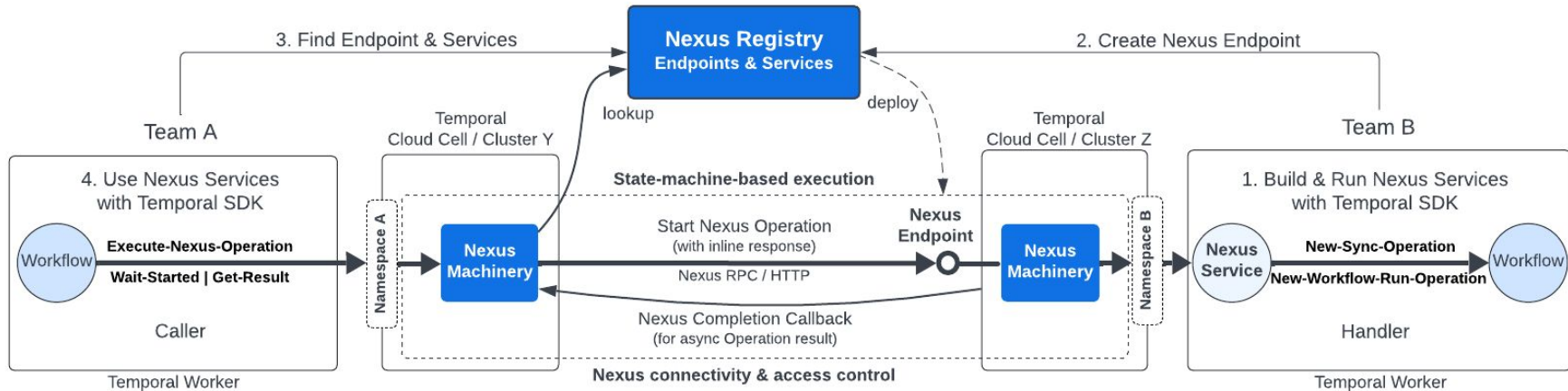
So, let's break this up



There are options for calling sub-agents:

- Call sub-agent via activity
- Sub-agent via child workflow
- Sub-agent via Nexus

Nexus



<https://docs.temporal.io/nexus>



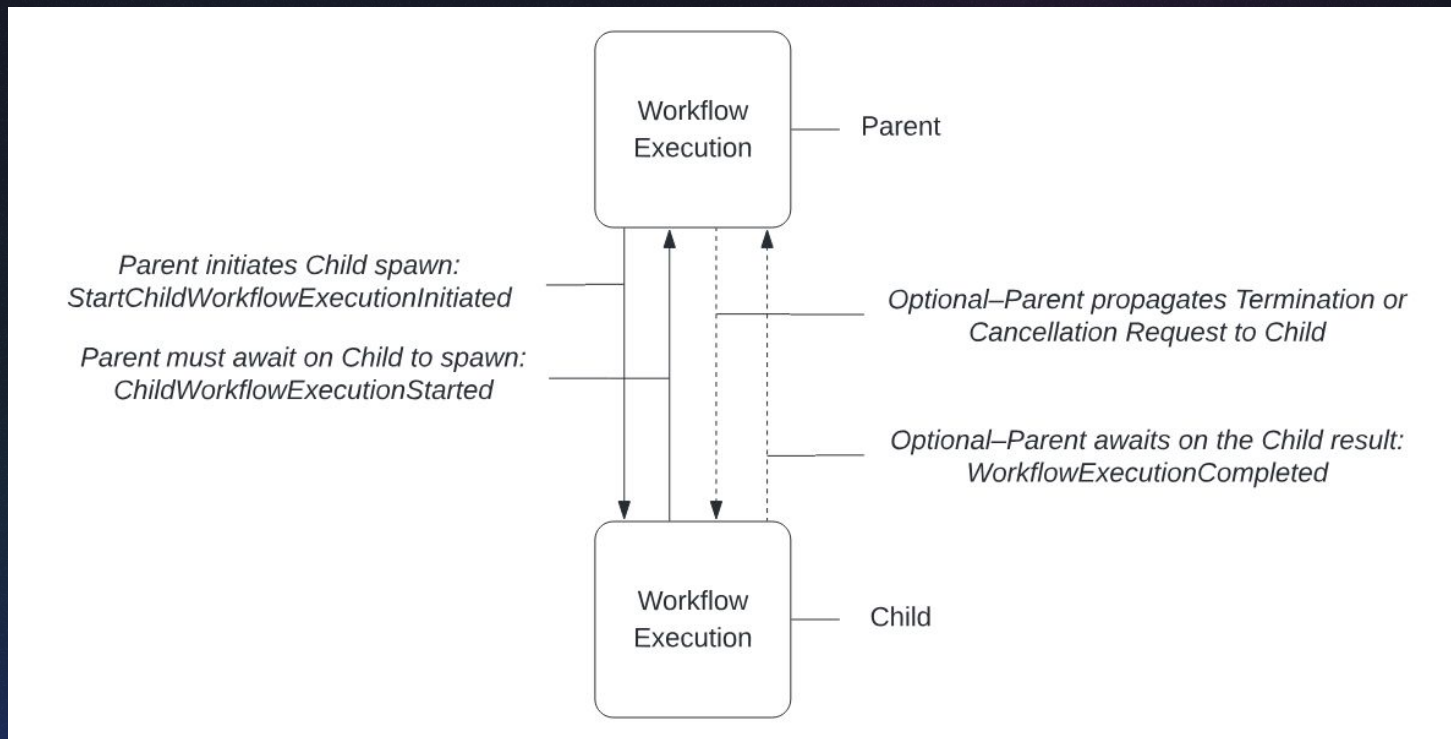
DEMO #5A

Multi-agent orchestration: Nexus

TBD



Child Workflows:



DEMO #5B

Multi-agent orchestration: Child Workflows

TBD



Child workflow

Pros

- Same team, one deploy
- Direct signal + query handle
- Built-in parent-close policy
- Lower latency, no endpoint setup
- Traces propagate today

Cons

- Same namespace, same cluster
- Tight type coupling (both sides import the class)
- Sync result only
- Python-only contract

Nexus

Pros

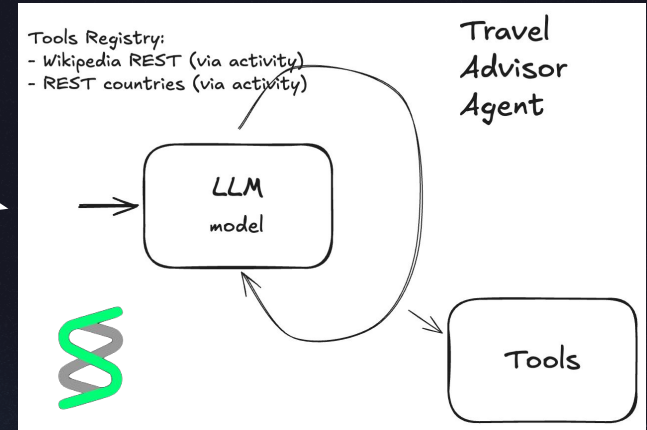
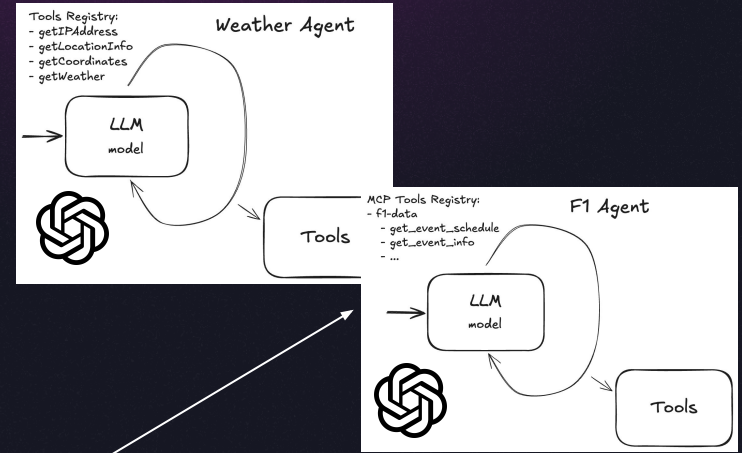
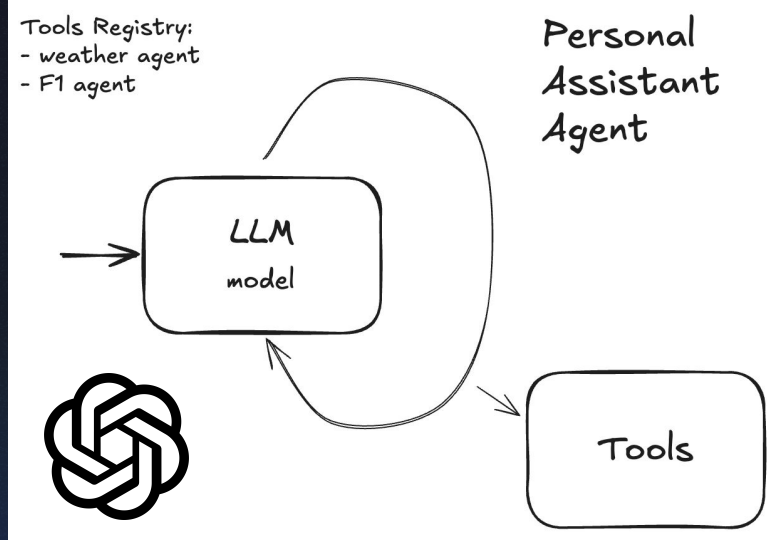
- Crosses namespaces / clusters
- Stable, language-agnostic contract
- Independent deploy cadence
- Endpoint-level policy hooks
- Async completion supported
- Signal/query support via Nexus

Cons

- Endpoint registration per service
- Extra hop latency
- Trace context fragmentation (current Python SDK)



Heterogeneous Agents



DEMO #6

Multi-agent orchestration: Heterogeneous Agents

TBD



RECAP

- ✓ Foundations of Agentic Systems
- ✓ Building a Durable Agent from Scratch
- ✓ Building with an Agent Framework, but Durable
- ✓ Adding MCP Tools
- ✓ Humans in the Loop (HITL)
- ✓ Orchestrating (micro)Agents



FEEDBACK



<https://t.mp/replay26-ws-feedback>



YOU CAN FIND ALL OF THIS HERE

Exercises/Demos



<https://t.mp/ai-workshop-python>

OpenAI Agents SDK/Temporal Integration



<https://t.mp/oai-agent>



